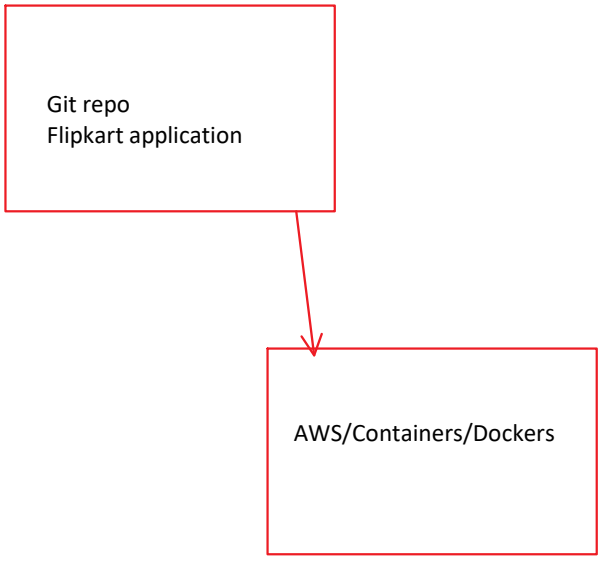
18 April 2026 14:15

```
import sys
text = sys.stdin.read()
print(text.upper())
```



The read() is reads the all incoming input until EOF(end of Line):

Piped text
Redirected file input
Manual typing until end signal (CTL+Z) then enter button

type payslip.tx | python .\day4.py--> windows os
echo payslip.tx | python .\day4.py ---> Linux Os

```
import sys
sys.stderr.write("Kubernetes connection failed\n") # used to seperate error
message as log file

We can log the errors by using stderr.write() .. In to other files
```

PS D:\Myworkspace> python .\day4.py >kbs.log 2>kbserr.log

Here kbs.log will store only success messages
Kbserr.log file can store error messages which are thrown by sys,stderr.wrirte()

```
print("Starting deployment")
sys.stderr.write("Database connection warning\n")
sys.stderr.write("server is not running \n")
print("Deployment complete")
print("servers are running ")
```

Diff -- sys and print()

1. **Print is only one tool. Direct stream**
2. **Better control**
3. **Shell redirection support**
4. **Separate errors from normal output**
5. **Integrate with pipelines**
6. **Machine-readable automation output**

Stdin	**Standard**	**Receive the data from input**
Stdout	**Standard output**	**Normal output**
Stderr	**Standard error**	**Errors/warning**

Subprocess modules:
- Run shell commands
- Automate server task

- Install packages
- Backup files
- Call another script
- Read command output
- Monitor processes
- Handle errors

Import subprocess

```
subprocess.run("echo Hello Student", shell=True)
Echo is shell command which is used to write the data
```

Subprocess.open()

Starts process without waiting :
Used for
Background application
Long running task
Streaming data

```
subprocess.Popen("calc.exe")
subprocess.Popen("notepad.exe")
print("caluculator opened")
import revision
print(revision.gross)
```

```
Subprocess.check_output()
```

```
# output=subprocess.check_output(["python", "--version"],text=True)
# print(output)
# output=subprocess.check_output(["ping", "google.com"],text=True)
# print(output)
```

Subprocess.PIPE:-
Used when we want to cpture
Stdout
Stderr
Stdin

```
process=subprocess.Popen(
    ["git","--version"],
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE,
    text=True
)
out,err=process.communicate()
print(out,err)

We can also run the script of another file from current working directory
subprocess.run(["python",".\day1.py"])
```

Logging module:-
Recording the messages about a programs execution while running it
Its is a standard way to record error

messages,digonise issues, monitor system
Instead of using
Print("error")

We can use logging module :-

Logging gives :-
- Severity level
- Time stamp
- File output
- Filtering the data
- Structured records
- Production monitoring
- Error tracking'

Why use logging instead of print()?

Print()	LOGGING
Simple output	Professional monitoring
No levels	DEBUG,INFO,WARMING,ERROR
Hard to mange	Easy filtering
No timestamp	Timestamp
No file storage	Can save to file
Poor production	Best for production

Import logging:

Logging:warning("Low diskspace")

Logging levels

Level	Purpose	Numeric value
DEBUGGING	Detailed technical information	10
INFO	General information	20
WARNING	Something unexpected	30
ERROR	Failure happened	40
CRITICAL	Program may crash	50